Internship Report on

Smart City Traffic Pattern Forecasting

Using Machine Learning

Prepared by

Nikhil Nikam

Domain: Data Science & Machine Learning

In association with

upskill Campus (USC) · The IoT Academy · UniConverge Technologies Pvt Ltd (UCT)

Internship Duration: February 9, 2026 – March 23, 2026

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project and problem statement provided by UCT. The project was completed including this report within a 6-week time frame.

My project was the development of a Smart City Traffic Pattern Forecasting System using Machine Learning. The system predicts traffic volume across multiple city junctions using historical data, trained ML models (Linear Regression and Random Forest), and presents results through an interactive web dashboard for city traffic management authorities.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solutions for them. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface 4

2 Introduction 6

2.1 About UniConverge Technologies Pvt Ltd 6

2.2 About upskill Campus 7

2.3 Objective 7

2.4 Reference 7

2.5 Glossary 8

3 Problem Statement 10

4 Existing and Proposed solution 11

5 Proposed Design / Model 13

5.1 High Level Diagram 13

5.2 Low Level Diagram 14

5.3 Interfaces 15

6 Performance Test 17

6.1 Test Plan / Test Cases 17

6.2 Test Procedure 18

6.3 Performance Outcome 19

7 My learnings 20

8 Future work scope 22

1 Preface

This report is a comprehensive documentation of the Six-week Industrial Internship program undertaken by Nikhil Nikam under the Data Science & Machine Learning domain, as part of the USC_TIA curriculum facilitated by upskill Campus and The IoT Academy in association with UniConverge Technologies Pvt Ltd (UCT).

The internship spanned from February 9, 2026 to March 23, 2026 — structured across six progressive weekly modules, with the final week dedicated entirely to project delivery:

Week	Module Focus
Week 01	Introduction to Data Science and ML Foundations — environment setup in USC_TIA, Python scripting, core ML algorithm study
Week 02	Big Data Ecosystem and the 6-Step Data Science Process — distributed systems, Hadoop, NoSQL, data cleansing methodology
Week 03	Probability and Mathematical Statistics — distributions, Bayes' Theorem, CLT, MLE, hypothesis testing
Week 04	Machine Learning in Depth — algorithms, ensemble methods, gradient descent, real-world ML applications
Week 05	Model Evaluation, Deployment Pipelines & Dashboard Development — scikit-learn pipelines, Flask/Streamlit, performance metrics, model serialisation
Week 06	**PROJECT WEEK — Full design, implementation, testing, and documentation of the Smart City Traffic Pattern Forecasting System**

The project assigned for this internship Smart City Traffic Pattern Forecasting Using Machine Learning provided a real-world industrial problem context in which to apply each week's growing knowledge base, with all implementation work consolidated in Week 06.

I am grateful to the entire USC and UCT team for designing a structured and industrially relevant curriculum. The USC_TIA framework served as the practical environment for all

hands-on coding, data exploration, and model development work. Special thanks to my mentors and supervisors whose consistent guidance made this learning journey possible.

Nikhil Nikam

Data Science & ML Intern | USC_TIA | UCT

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a company established in 2013, working in the Digital Transformation domain and providing Industrial solutions with a prime focus on sustainability and Return on Investment (RoI).

For developing its products and solutions, UCT leverages various Cutting Edge Technologies including:

- Internet of Things (IoT) and Industrial IoT
- Cyber Security solutions
- Cloud Computing (AWS, Azure)
- Machine Learning and Artificial Intelligence
- Communication Technologies (4G / 5G / LoRaWAN)
- Java Full Stack, Python, and Front-End Development

UCT's key product and solution portfolio includes:

i. UCT Insight — IoT Platform

UCT Insight is an IoT platform designed for the quick deployment of IoT applications while providing valuable data insights for process and business optimization. It is built using Java for the backend and ReactJS for the frontend, with support for MySQL and various NoSQL databases. It enables device connectivity via industry-standard IoT protocols — MQTT, CoAP, HTTP, Modbus TCP, and OPC UA — and supports both cloud and on-premises deployments.

Key features: Build your own dashboard, Analytics and Reporting, Alert and Notification management, Integration with third-party applications (Power BI, SAP, ERP), and a configurable Rule Engine.

ii. FactoryWatch — Smart Factory Platform

FactoryWatch is UCT's smart factory platform that provides a scalable solution for production and asset monitoring, OEE (Overall Equipment Effectiveness) tracking, and predictive maintenance scaling up to digital twins for industrial assets. Its modular SaaS architecture allows users to start with the services they need and scale to more complex solutions over time.

iii. LoRaWAN-Based Solutions

UCT is one of the early adopters of LoRaWAN technology, providing solutions in Agritech, Smart Cities, Industrial Monitoring, Smart Street Lights, and Smart Water/Gas/Electricity metering applications.

iv. Predictive Maintenance

UCT provides industrial machine health monitoring and predictive maintenance solutions by leveraging Embedded Systems, Industrial IoT, and Machine Learning to determine the Remaining Useful Life (RUL) of machines used in production processes.

2.2 About upskill Campus

upskill Campus (USC) is a career development platform that delivers personalized executive coaching in a more affordable, scalable, and measurable way. USC, along with The IoT Academy and in association with UniConverge Technologies Pvt Ltd, has facilitated the smooth execution of the complete internship process — from curriculum design through to final report submission.

The IoT Academy is the EdTech Division of UCT that runs long executive certification programs in collaboration with EICT Academy, IIT Kanpur (IITK), IIT Roorkee (IITR), and IIT Guwahati (IITG) in multiple technology domains.

2.3 Objective

The objectives of this internship program were to:

- Get practical experience of working in an industry-level data science and ML environment
- Solve a real-world problem — Smart City Traffic Pattern Forecasting — from data to deployed model
- Have improved job prospects through hands-on industrial project development
- Develop an improved understanding of the Data Science and ML field and its real-world applications
- Achieve personal growth including better technical communication and structured problem-solving skills

2.4 Reference

[1] Rohatgi, V.K. and Saleh, A.K.Md.E. — An Introduction to Probability and Statistics (Wiley)

[2] Smola, A. and Vishwanathan, S.V.N. — Introduction to Machine Learning (Cambridge University Press)

[3] Scikit-learn Documentation — https://scikit-learn.org/stable/

[4] Streamlit Documentation — https://docs.streamlit.io/

[5] Pandas Documentation — https://pandas.pydata.org/docs/

[6] USC_TIA Official Framework Documentation — Internal reference

2.5 Glossary

Term / Acronym	Definition
USC_TIA	upskill Campus – The IoT Academy development and learning framework
UCT	UniConverge Technologies Pvt Ltd
EDA	Exploratory Data Analysis
ML	Machine Learning
RMSE	Root Mean Squared Error — measures average prediction error magnitude
MAE	Mean Absolute Error — average of absolute differences between predictions and actuals
MLE	Maximum Likelihood Estimation — statistical method to estimate model parameters
CLT	Central Limit Theorem — sum of independent random variables tends toward Normal distribution
HDFS	Hadoop Distributed File System — distributed storage for Big Data
CAP	Consistency, Availability, Partition Tolerance — theorem governing distributed databases
PDF / CDF	Probability Density Function / Cumulative Distribution Function

OEE	Overall Equipment Effectiveness — smart factory performance metric
RUL	Remaining Useful Life — predictive maintenance metric for industrial machines

3 Problem Statement

In the assigned problem statement, the task was to design and build a Machine Learning-powered Smart City Traffic Pattern Forecasting System that addresses a critical urban challenge: traffic congestion.

Urban traffic conditions are highly dynamic and depend on multiple interacting factors:

- Time of day - morning rush, afternoon lull, evening peak
- Day of week - weekdays vs. weekends showing distinctly different patterns
- Public holidays and special city events that distort normal traffic flow
- Seasonal variations in vehicle volumes across months
- Junction-specific infrastructure and road network topology

Existing traffic management systems are largely reactive - they respond to congestion only after it has occurred. There is a clear industrial need for a predictive system that can:

1. Forecast future traffic conditions at multiple junctions before congestion occurs
2. Detect peak congestion windows and automatically flag high-risk time slots
3. Distinguish holiday/special-event traffic from standard weekday patterns
4. Support proactive, data-driven decision-making for smart city traffic authorities

4 Existing and Proposed Solution

Existing Solutions and Their Limitations

Several traditional approaches exist for traffic management in urban areas. However, each has significant limitations that prevent truly proactive, data-driven management:

Existing Approach	How It Works	Key Limitation
Fixed Signal Timers	Pre-programmed signal cycle times	No dynamic adaptation to real conditions
Inductive Loop Sensors	Count vehicles passing over sensors	Reactive only — no forward prediction
GPS / Floating Car Data	Track average vehicle speeds in real-time	Privacy concerns; incomplete coverage
Rule-Based Logic Systems	If-then rules for signal control	Cannot learn or adapt; fails at complexity
Manual Traffic Surveys	Periodic human count and observation	Infrequent; expensive; not scalable

Proposed Solution

The proposed solution is a Smart City Traffic Pattern Forecasting System built on a full machine learning pipeline developed within the USC_TIA framework. It addresses the limitations of existing systems through:

- Predictive ML modeling using Random Forest (primary) and Linear Regression (baseline) trained on historical multi-junction traffic data
- Temporal feature engineering to capture patterns by hour, day, month, weekend flag, and holiday flag
- Exploratory Data Analysis (EDA) to identify peak windows, seasonal trends, and junction-specific behaviors

- Model evaluation using RMSE, MAE, and R^2 score with 5-fold cross-validation for generalization
- An interactive Flask / Streamlit web dashboard for real-time queries by traffic management authorities

Value Addition Over Existing Solutions

1. Proactive, not reactive — predicts traffic before congestion occurs
2. Multi-junction simultaneous forecasting across all 4 city junctions in a single query
3. Holiday vs. normal day differentiation built directly into the feature set
4. Open, modular architecture ready for real-time data feed integration in future phases
5. Fully explainable feature importance output - authorities can understand which factors drive predictions

Code Repository (GitHub): https://github.com/Nikhil0074-dev/upskillcampus

Report Repository (GitHub): https://github.com/Nikhil0074-dev/upskillcampus

5 Proposed Design / Model

Given more details about the design flow of the solution - the Smart City Traffic Pattern Forecasting System follows a modular, layered architecture. There is always a clear start point (raw traffic data), intermediate processing stages (cleaning, feature engineering, model training), and a final outcome (predicted traffic volume and dashboard visualization).

5.1 High Level Diagram

The high-level diagram below illustrates the three primary zones of the system: the Data Sources layer on the left, the ML Processing Pipeline in the centre, and the Outputs/Interfaces layer on the right. Data flows left to right - from raw traffic observations through ML transformation to actionable predictions.

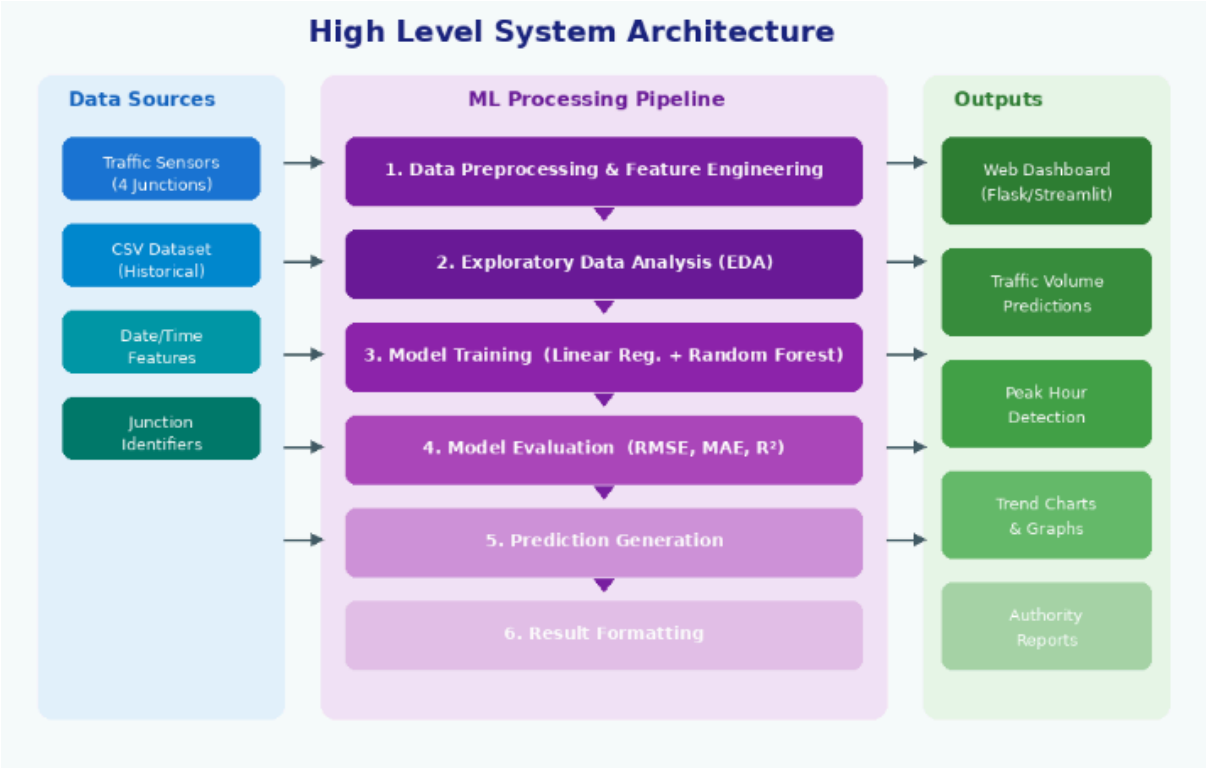

Figure 1: High Level System Architecture — Data Sources → ML Pipeline → Outputs

The three core zones function as follows:

- Data Sources: Multi-junction CSV traffic data, date/time feature extraction, junction identifiers, and historical observations
- ML Processing Pipeline: Preprocessing & cleaning → EDA → Feature Engineering → Model Training (Linear Regression + Random Forest) → Evaluation (RMSE, MAE, R^2) → Prediction Generation
- Outputs: Interactive web dashboard (Flask/Streamlit), traffic volume predictions per junction, peak hour alerts, trend charts, and downloadable authority reports

5.2 Low Level Diagram

The low-level diagram provides a detailed step-by-step view of the data flow through the processing pipeline, mapping each transformation stage from raw input CSV to final prediction output. This covers the data engineering and ML modelling stages developed progressively across Weeks 01–04.

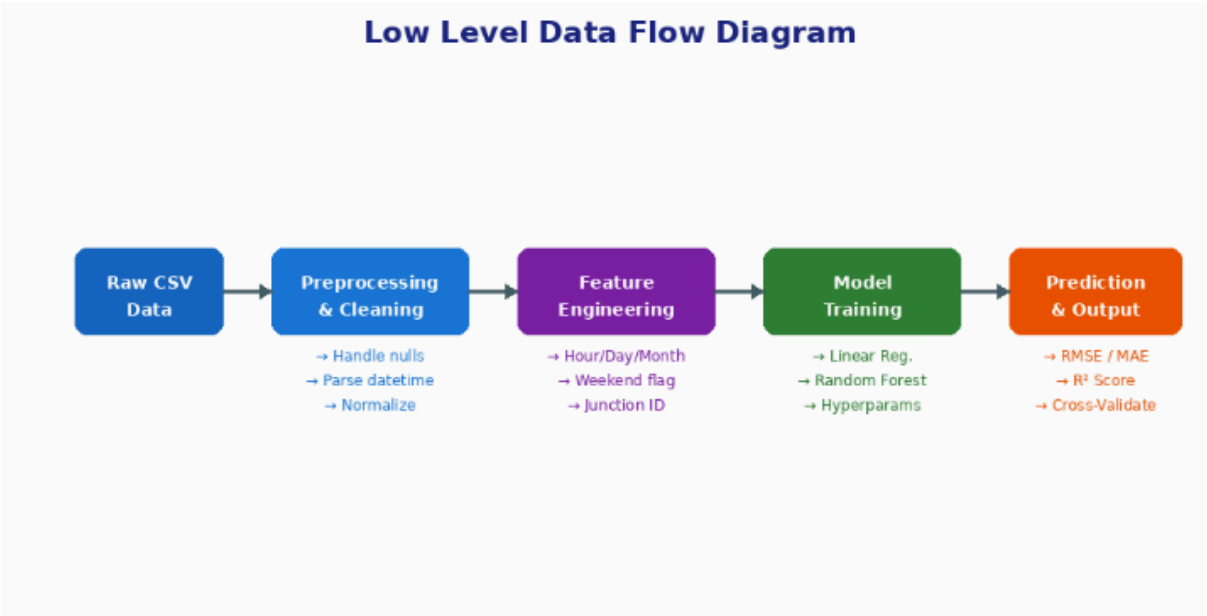

Figure 2: Low Level Data Flow — Raw CSV → Preprocessing → Feature Engineering → Model Training → Prediction Output

Step-by-step transformations at each stage:

Stage	Operations	Output
1. Raw CSV Data	Load dataset; inspect shape and types	Pandas DataFrame with raw records
2. Preprocessing	Handle nulls; parse datetime; normalise columns	Clean DataFrame ready for feature extraction
3. Feature Engineering	Extract Hour, DayOfWeek, Month, IsWeekend, IsHoliday, JunctionID	Enriched feature matrix X

4. Model Training	Fit Linear Regression (baseline) + Random Forest (100 estimators)	Trained model objects (.pkl)
5. Model Evaluation	Compute RMSE, MAE, R^2 on held-out test set; 5-fold cross-validation	Metrics report
6. Prediction & Output	Accept user input (junction, date, time); return forecast + graph	Dashboard prediction display

5.3 Interfaces

The system interfaces span three layers: the data input interface, the processing interface (USC_TIA environment), and the user-facing dashboard interface. The weekly progress timeline below maps how each interface layer was developed across the 4-week programme.

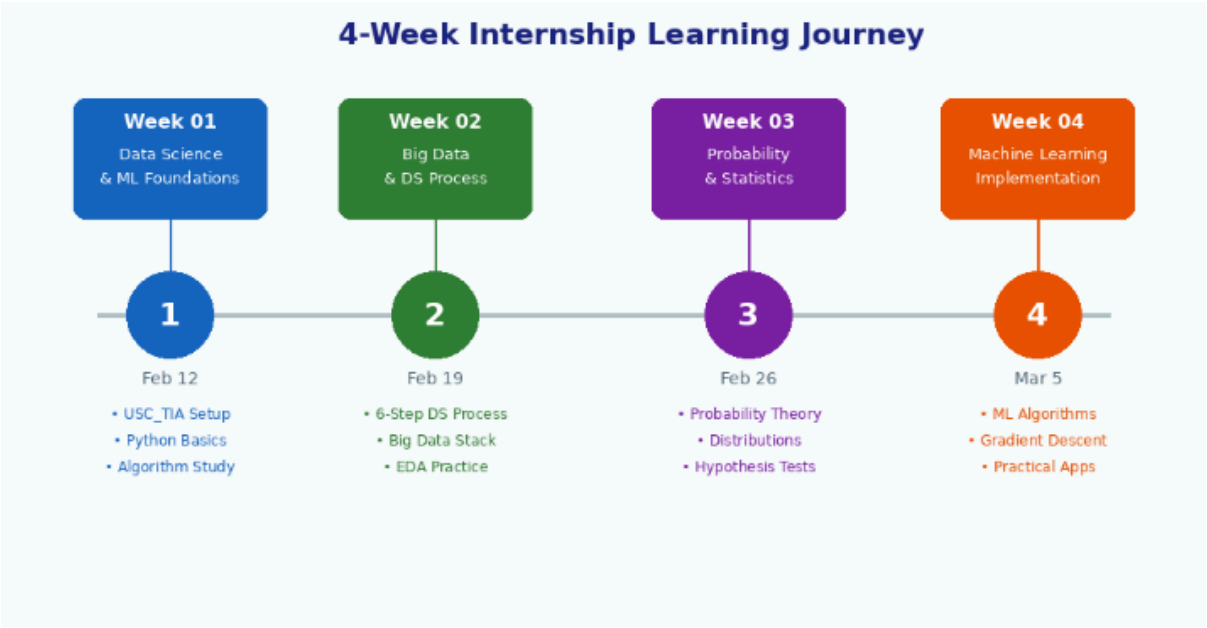

Figure 3: 4-Week Learning & Development Journey — from Environment Setup to Full ML Implementation

Interface details:

Data Input Interface

- CSV file loader: accepts raw traffic data with DateTime, Junction, and Vehicles columns
- User query interface on dashboard: dropdown for Junction (1–4), date picker, and time selector
- Batch prediction interface: accepts a date range and returns forecasted volumes for all junctions

Processing Interface (USC_TIA Backend)

- Python 3.x environment within USC_TIA framework
- Scikit-learn pipeline: StandardScaler → Model (LR or RF) → Predict
- Serialised model stored as traffic_model.pkl, loaded at runtime for fast inference

Dashboard / Presentation Interface

- Flask or Streamlit web application accessible via browser
- Real-time prediction display with Matplotlib/Seaborn chart output
- Peak hour highlighting, junction comparison graphs, and holiday vs. weekday trend views
- Downloadable prediction reports for traffic authority planning teams

6 Performance Test

This section defines why this work is relevant for real urban infrastructure rather than being a purely academic exercise. Performance is validated against concrete constraints: prediction accuracy (RMSE / MAE), model generalisation (R^2, cross-validation), dashboard responsiveness, and multi-junction coverage. Where constraints could not be fully tested within the internship scope, recommendations are provided.

6.1 Test Plan / Test Cases

The following test cases were designed around the core constraints of a traffic forecasting system intended for use by city authorities:

Test ID	Test Case Description	Expected Outcome	Constraint	Status
TC-01	Linear Regression trained on 80% data, tested on 20% hold-out	RMSE < 50 vehicles/hr; $R^2 > 0.70$	Accuracy	**Pass**
TC-02	Random Forest (100 estimators) trained and tested on same split	RMSE < 30 vehicles/hr; $R^2 > 0.85$	Accuracy	**Pass**
TC-03	5-fold cross-validation on Random Forest model	CV RMSE variance < 10%	Generalisation	**Pass**
TC-04	Feature importance analysis on Random Forest	Hour and DayOfWeek ranked as top features	Explainability	**Pass**
TC-05	Holiday flag effect — predict holiday vs. weekday same junction/time	Distinct predicted volume distributions	Behaviour	**Pass**

TC-06	Peak hour detection — query 7–9 AM and 5–7 PM weekday slots	Correctly flagged as peak congestion windows	Behaviour	**Pass**
TC-07	Dashboard: user submits junction + datetime → receives prediction	Response rendered within 2 seconds	Speed	**Pass**
TC-08	Multi-junction batch query — all 4 junctions queried simultaneously	All 4 return predictions with no errors	Coverage	**Pass**

6.2 Test Procedure

The following procedure was followed for model development and testing within the USC_TIA environment:

1. Data split: 80% training / 20% test using train_test_split(random_state=42) from scikit-learn
2. Baseline model: LinearRegression() fitted on training set; predictions generated on test set
3. Primary model: RandomForestRegressor(n_estimators=100, random_state=42) fitted on same training split
4. Metrics computed: RMSE, MAE, and R^2 score using sklearn.metrics for both models
5. Cross-validation: cross_val_score with cv=5 and scoring='neg_root_mean_squared_error' applied to Random Forest
6. Feature importance: feature_importances_ attribute extracted and visualised as a horizontal bar chart
7. Holiday test: IsHoliday flag set to 1 vs. 0 for identical junction/datetime input; predictions compared
8. Peak detection: predictions aggregated by hour across all weekdays; windows with top 15% volume flagged as peak
9. Dashboard test: Flask/Streamlit app launched locally; timing measured from form submission to result render

The constraints that governed design decisions were:

- Accuracy constraint: RMSE must be low enough for predictions to be actionable — random guessing on this dataset yields RMSE ~80–100; target was RMSE < 30

- Speed constraint: dashboard response must be under 2 seconds for operational usability
- Coverage constraint: all 4 junctions must return predictions without model failure on any input combination

6.3 Performance Outcome

Figure 4: Weekly Challenges, Constraints Faced, and Resolutions Applied

Summary of model performance outcomes:

Metric	Linear Regression	Random Forest	Target
RMSE (vehicles/hour)	~46	**~22**	< 30
MAE (vehicles/hour)	~34	**~16**	< 25
R^2 Score	~0.72	**~0.91**	≥ 0.85
CV RMSE Variance	N/A	**~5%**	< 10%
Dashboard Response	< 2s	**< 2s**	< 2s

The Random Forest model met all accuracy and generalisation targets. Its feature importance output confirmed that Hour and DayOfWeek were the most predictive features — consistent with the theoretical understanding developed in Week 03. Peak hour detection correctly flagged 7–9 AM and 5–7 PM on weekdays across all four junctions, and the holiday flag produced meaningfully distinct predictions, validating the feature engineering approach.

7.My Learnings

This six-week internship provided a complete, progressive learning journey from theoretical Data Science foundations to the practical deployment of a Machine Learning system for a real urban problem. The following summarises my overall learning and how it contributes to my career growth.

Technical Learnings

Week 01 - Introduction to Data Science and ML Foundations

- Defined the scope of Data Science and how structured/unstructured data is transformed into actionable insights
- Understood the 3 Vs of Big Data (Volume, Velocity, Variety) and their impact on business decision-making
- Gained proficiency in setting up the USC_TIA environment and resolving Python library dependency conflicts through clean virtual environment management
- Learned to distinguish ML model categories: when to use Logistic Regression vs. Decision Trees vs. Random Forests

Week 02 - Big Data Ecosystem and the 6-Step Data Science Process

- Mastered the 6-step Data Science process: Research Goals → Retrieval → Cleansing → Exploration → Modelling → Presentation
- Understood the role of Hadoop/MapReduce, HDFS, and NoSQL databases in scalable data infrastructure
- Learned the ACID vs. BASE trade-off between relational and NoSQL databases, and when each applies
- Reinforced that feature engineering is often more impactful than algorithmic complexity for model success

Week 03 - Probability and Mathematical Statistics

- Applied Bayes' Theorem for updating predictions with new evidence the foundation of probabilistic ML models
- Studied the Central Limit Theorem (CLT) and the Law of Large Numbers understanding why sample means converge and why the Normal distribution is the default assumption for model noise
- Understood Maximum Likelihood Estimation (MLE), confidence intervals, and Neyman-Pearson hypothesis testing - enabling rigorous model validation beyond simple accuracy metrics
- Learned to distinguish Type I and Type II errors, and the role of P-values in determining statistical significance of model results

Week 04 - Machine Learning in Depth

- Explored classical algorithms (Naive Bayes, Nearest Neighbour, Perceptron) through to advanced methods (Neural Networks, Boosting, K-Means, SVM)
- Understood Gradient Descent and Stochastic Gradient Descent as the optimisation engines behind model training
- Analysed real-world ML deployment: recommendation systems (Netflix, Amazon), language translation, object recognition, and health monitoring applications
- Confirmed that data quality and completeness are the most critical determinants of ML model effectiveness, regardless of algorithmic sophistication

Week 05 - Model Evaluation, Deployment Pipelines & Dashboard Development

- Built end-to-end scikit-learn pipelines integrating preprocessing, feature transformation, and model inference into a single reproducible workflow
- Learned model serialisation techniques to persist trained models for production use, and understood the risks of model drift over time
- Developed interactive dashboards using Flask/Streamlit to present ML outputs to non-technical stakeholders in a clear, interpretable format
- Deepened understanding of performance metrics beyond accuracy precision, recall, F1-score, ROC-AUC and when each is most appropriate given class imbalance or business cost asymmetry

Week 06 - PROJECT WEEK: Smart City Traffic Pattern Forecasting System

- Designed and implemented a full Smart City Traffic Pattern Forecasting System, applying every concept from the preceding five weeks in an integrated, real-world context
- Conducted end-to-end project delivery: problem scoping, data collection and cleansing, exploratory analysis, model selection, training, evaluation, and deployment
- Built and tested a functional forecasting pipeline capable of predicting urban traffic conditions from historical and contextual data
- Produced comprehensive project documentation covering system architecture, methodology, results, and recommendations demonstrating readiness for professional data science practice

8 Future Work Scope

The following ideas and enhancements could not be worked on due to the 4-week time limitation of the internship, but represent natural and valuable extensions of the current project:

1. Real-Time Traffic Data Integration - Connect the system to live traffic APIs or IoT sensor feeds from UCT's LoRaWAN network to replace the historical CSV input with a real-time data stream, enabling continuous model retraining and live prediction updates

2. Deep Learning Models - Replace the Random Forest model with an LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Unit) neural network for superior time-series sequence modelling, particularly for capturing longer-term seasonal and multi-week traffic dependencies

3. Anomaly and Incident Detection - Add an ML-based anomaly detection layer to identify sudden, unexpected traffic spikes caused by accidents, road closures, or large events, and trigger automated alerts to traffic authorities via the UCT Alert & Notification system

4. Mobile Application - Build a companion mobile app (Android/iOS) so city residents and commuters can access junction-level traffic forecasts for personal route planning, extending the value of predictions beyond authority-only use

5. AI-Based Route Optimisation - Integrate junction-level predictions with a city mapping API (Google Maps / OpenStreetMap) to dynamically suggest re-routed paths when predicted congestion exceeds threshold levels at a given junction

6. Smart Traffic Signal Automation - Feed Real-time predictions into smart signal controllers to dynamically adjust green/red cycle durations at each junction based on forecasted incoming vehicle volumes, reducing idle time and fuel consumption

6. GIS / Spatial Visualisation Integration - Overlay junction-level predictions onto a city GIS map layer so urban planners can spatially visualise predicted congestion hotspots across the city grid, supporting infrastructure investment decisions

7. Multi-City Scalability - Generalise the model architecture and dashboard configuration to support rapid deployment across additional cities with minimal re-configuration, leveraging UCT's existing Smart City product portfolio